



جلسه سوم: تسلط بر RAG

طراحی معماری داده‌ها، ابزارهای آماده، و تکنیک‌های پیشرفته ایجنتیک

مدیریت یکپارچه دانش خارجی برای مدل‌های زبانی

مفهوم RAG چیست؟



تعریف: RAG (Retrieval-Augmented Generation) چارچوبی است که به مدل‌های زبانی اجازه می‌دهد قبل از تولید پاسخ، اطلاعات دقیق و به‌روز را از یک پایگاه داده خارجی استخراج کنند.

- **پرسش (Query):** دریافت درخواست کاربر.
- **بازیابی (Retrieval):** یافتن اسناد مرتبط از پایگاه داده (Vector DB).
- **تزریق (Augmentation):** ترکیب اسناد با درخواست اصلی کاربر.
- **تولید (Generation):** ساخت پاسخ دقیق توسط مدل هوش مصنوعی.

چرا به RAG نیاز داریم؟



اطلاعات بلادرنگ (Real-time)

برخلاف مدل‌های ثابت، RAG همیشه به جدیدترین تغییرات داده‌های شما دسترسی دارد.



دسترسی به داده‌های خصوصی

امکان خواندن کدهای پرایوت، دیتابیس‌های لوکال سازمانی، تاریخچه تیکت‌ها و ایمیل‌های محرمانه.



حل مشکل توهم (Hallucination)

پاسخ‌ها مستند به منابع واقعی سیستم شما هستند و از حدسیات بی‌اساس مدل جلوگیری می‌شود.

ابزارهای آماده سازمانی (Off-the-shelf RAG)

راهکارهای Enterprise شرکت‌های بزرگ بدون نیاز به ساخت زیرساخت پیچیده:

پلتفرم	ابزار آماده	ویژگی کلیدی برای توسعه‌دهندگان
Google Gemini	Grounding / Vertex AI Search	دسترسی Native به سرچ گوگل و اسناد Workspace با کمترین تاخیر (Latency).
ChatGPT	Azure OpenAI On Your Data	یکپارچگی عمیق با اکوسیستم مایکروسافت، پشتیبانی از آپلود مستقیم و سرچ هیبریدی.
Claude	Claude Projects	تخصیص فضای کاری (Workspace) با کانتکست بسیار بالا، عالی برای تحلیل کدهای حجیم.

رویکرد DeepSeek در RAG

قدرت در سفارشی‌سازی

دیپ‌سیک (DeepSeek) برخلاف گوگل و مایکروسافت، پلتفرم ابری سازمانی با ابزارهای RAG آماده (Drag & Drop) ارائه نمی‌دهد.

- **مزیت اصلی:** API بسیار ارزان و عملکرد استدلالی بالا (در مدل‌های V3 و R1).
- **شیوه استفاده:** ایده‌آل برای ساخت پایپ‌لاین‌های اختصاصی RAG با LangChain و LlamaIndex.
- توسعه‌دهندگان کنترل کامل روی پایگاه داده برداری (Vector DB) خود دارند.



Developer-First

عدم وجود پنل آماده به معنای نیاز به کدنویسی بک‌اند است، اما هزینه نگهداری سیستم را به شدت کاهش می‌دهد.

معماری ساخت RAG اختصاصی (Custom Architecture)

۴. Orchestration

استفاده از LangChain
برای مدیریت جریان
داده بین دیتابیس و
مدل زبانی.

۳. Vector DB

ذخیره‌سازی بردارها در
دیتابیس‌های تخصصی
مانند ChromaDB،
Pinecone یا Milvus.

۲. Embedding

تبدیل متن‌های خرد
شده به بردارهای عددی
توسط مدل‌هایی مثل
text-embedding-004
گوگل.

۱. Ingestion

دریافت داده‌ها
(PDF, API, DB) و خرد
کردن آن‌ها به قطعات
کوچکتر به نام Chunk.

چالش‌های پیاده‌سازی RAG در Production

استراتژی Chunking

اگر قطعات خیلی کوچک باشند مفهوم از دست می‌رود؛ اگر خیلی بزرگ باشند، هزینه توکن بالا رفته و دقت LLM کاهش می‌یابد.

کیفیت بازیابی (Retrieval)

پیدا نکردن سند درست در دیتابیس. راه‌حل: استفاده از جستجوی هیبریدی (BM25 + Vector) و ابزارهای Reranking.

امنیت و دسترسی (Permissions)

اطمینان از اینکه کاربر فقط به اسنادی (مثلاً فیش حقوقی) دسترسی دارد که در سیستم اصلی مجاز به دیدن آنهاست.

مقایسه استراتژیک: RAG در برابر Fine-Tuning

ویژگی مورد نیاز	RAG (بازیابی اطلاعات)	Fine-Tuning (تنظیم دقیق)
هدف اصلی	تزریق دانش جدید و دیتای متغیر	تغییر لحن، استایل و فرمت خروجی (مثلا JSON)
سرعت آپدیت داده‌ها	بلادرنگ (فقط دیتابیس آپدیت می‌شود)	بسیار کند (نیاز به آموزش مجدد مدل)
جلوگیری از توهم	بسیار بالا (به دلیل ارجاع مستقیم به منبع)	پایین (مدل همچنان ممکن است حدس بزند)
هزینه پیاده‌سازی	ارزان (فقط هزینه توکن ورودی و دیتابیس)	بسیار گران (هزینه سرورهای GPU برای Training)
شفافیت (Citation)	دارد (ارائه لینک دقیق منبع به کاربر)	ندارد (دانش در وزن‌های شبکه مخفی است)

چه زمانی از کدام روش استفاده کنیم؟

از Fine-Tuning استفاده کنید اگر:

- مدل باید ادبیات تخصصی یک صنعت را بیاموزد.
- همیشه به یک فرمت خروجی خاص (مثلا ساختار پیچیده کد) نیاز دارید.
- نیاز به تغییر شخصیت (Persona) مدل دارید.

از RAG استفاده کنید اگر:

- داده‌های شما مدام تغییر می‌کنند (قیمت، موجودی).
- نیاز دارید سیستم به منبع پاسخ ارجاع دهد (Citation).
- حجم اطلاعات متنی شما بسیار بالاست.

حالت ایده‌آل سازمان‌ها (Hybrid): ترکیب مدل Fine-Tune شده با یک پایپ‌لاین RAG!

بخش دوم: تکنیک‌های پیشرفته

فراتر از جستجوی برداری ساده (Naive RAG)؛ پیش‌نیاز ورود به دنیای ایجنت‌ها.

تکنیک پیشرفته ۱: Graph RAG

ترکیب گراف دانش و جستجوی معنایی

در Graph RAG، به جای تبدیل ساده متن به بردار، مدل هوش مصنوعی موجودیت‌ها (Entities) و روابط (Relationships) آن‌ها را استخراج کرده و یک شبکه متصل می‌سازد.

- **مزیت کلیدی:** حل مشکل استدلال چندمرحله‌ای (Multi-hop Reasoning).
- مناسب برای زمان‌هایی که پاسخ در ده‌ها سند مختلف پخش شده است.

Graph RAG نمونه درک روابط در //

Node A: "میکروسرویس پرداخت"

Edge: "متکی است به"

Node B: "PostgreSQL دیتابیس"

با بررسی لاگ‌های نود A جستجوی گراف: یافتن دلیل قطعی سرویس B.

تکنیک پیشرفته ۲: Agentic RAG

Agentic RAG دقیقاً همان پلی است که ما را به
مباحث **LangChain** و **LangGraph** در جلسه
چهارم متصل می‌کند.

یک فرآیند خودمختار و استدلال‌گر

تبدیل RAG از یک "پایپ‌لاین خطی" به یک سیستم
تصمیم‌گیرنده، که توسط یک ایجنت مدیریت می‌شود.

- **مسیریابی (Routing):** تصمیم برای جستجو در وب،
دیتابیس، یا پاسخ مستقیم.
- **بازنویسی کوئری:** بهینه‌سازی سوال کاربر برای موتور
جستجو.
- **خوداصلاحی (Self-Correction):** ارزیابی اسناد
پیدا شده و جستجوی مجدد در صورت نیاز.

پایان جلسه سوم

شما اکنون معماری داده‌های سیستم‌های هوشمند را می‌شناسید.

🔗 آماده ورود به دنیای ارکستراسیون ایجنت‌ها با LangGraph هستید؟